

AI-Powered SOC Triage Lab

Comprehensive Build Documentation & Production Architecture

Consolidated System Engineering Report

Last Consolidated Revision: June 2026

Contents

1	Project Overview	2
1.1	Objective	2
1.2	Operational Architecture Status	2
1.3	Data Stream Topology	2
2	SIEM Platform Provisioning & Network Configurations	3
2.1	Kali Linux Host Setup	3
2.2	Remediating Interface Binding (No IP on VM)	3
2.3	Clean Installation Script	3
2.4	Wazuh Indexer Network Configuration Fix	3
3	Endpoint Agent & Adversary Simulations	4
3.1	Deploying Agent on Target Windows Host	4
3.2	Simulation Suite: Tactic-based Alerts Generation	4
3.3	Verification Checklist	5
4	Production n8n Pipeline Implementation	6
4.1	Node 1: Schedule Trigger	6
4.2	Node 2: Get Wazuh Token	6
4.3	Node 3: Get Alerts	6
4.4	Node 4: Parse Alerts (JS Code)	6
4.5	Node 5: Loop Over Items	7
4.6	Node 6: Local AI Triage	7
4.7	Node 7: Parse Verdict (JS Code)	7
4.8	Node 8: If (Conditional Escalation Override)	8
4.9	Node 9: HTTP Request (Discord Webhook)	8
5	Troubleshooting Registry & Stabilizations	9
5.1	The Metadata Field Depletion (Empty Fields / N/A Bug)	9
5.2	The 1-Item Batch Strangle	9
5.3	Ollama Streaming Crash (Go Unmarshal String Errors)	9
5.4	Llama 3.2 AD Account Deletion Hallucination	9
5.5	Google Gemini API Constraints	9

1 Project Overview

1.1 Objective

The objective of this engineering initiative is to design, deploy, and refine an automated Tier 1 SOC Analyst pipeline. The pipeline ingests telemetry logs from endpoint agents via a centralized Wazuh SIEM, structures and filters alerts within an n8n workflow, orchestrates automated deep-reasoning classification via a local LLM (Llama 3.2), and dispatches rich contextual notifications to incident responders over Discord.

1.2 Operational Architecture Status

Component	Technology / Specifications	Status
SIEM	Wazuh 4.12 on Kali Linux VM	Active / Listening
Endpoint Agent	Wazuh Windows Agent v4.12	Active / Logging
Orchestration	n8n Self-Hosted (v2.27.4 on Node.js)	Operational
Triage Engine	Llama 3.2 (3B Parameters) via local Ollama	Non-Streaming / Fixed
Triage Logic	Multi-Event Batch Splitting (n8n SplitInBatches)	Optimized
Notifications	Discord Webhook Integration	Active / Filtered
Data Store	OpenSearch (Indexer) at port 9200	Accessible

1.3 Data Stream Topology

To establish an optimal flow that avoids bottlenecking and accounts for multiple concurrent indicators of compromise (IoCs), the orchestration pipeline follows this sequence:

1. **Wazuh Agent Logs** → Real-time Event Collection
2. **Wazuh Indexer** → OpenSearch DB: Port 9200
3. **n8n Workflow Ingestion** → Poll Schedule: Every 50s
4. **Parse Alerts** → JavaScript Execution for Safe Ingestion
5. **Loop Over Items** → Batch size 5 (Resolves array bottleneck)
6. **Local AI Triage** → Non-streaming Llama 3.2 analysis
7. **Parse Verdict** → Telemetry Preservation (Restores IPs, Names)
8. **Conditional Override** → Rule Level ≥ 5 OR Verdict = escalate
9. **Discord Webhook** → Notification Dispatch

2 SIEM Platform Provisioning & Network Configurations

2.1 Kali Linux Host Setup

The Wazuh Manager is hosted inside a VirtualBox virtual machine running Kali Linux.

- **Primary VM Interface (eth0):** NAT Adapter (provides essential internet transit)
- **Secondary VM Interface (eth1):** Host-Only Adapter (VM-to-Host network segment)
- **Assigned Static IP (eth1):** 192.168.56.103
- **Wazuh Dashboard Port:** 443 (HTTPS)
- **Wazuh Manager API Port:** 55000
- **Wazuh Indexer (OpenSearch) Port:** 9200

2.2 Remediating Interface Binding (No IP on VM)

Symptom: Following initial VM instantiation, the adapter showed no assigned IPv4 address on the host-only interface.

Resolution Protocol:

1. Shut down the Kali guest OS.
2. In VirtualBox Manager: Select Kali VM → Settings → Network.
3. Under **Adapter 1:** Check "Enable Network Adapter" and bind to NAT.
4. Under **Adapter 2:** Check "Enable Network Adapter" and bind to Host-Only Adapter.
5. Boot VM and run `ip a` to verify eth1 shows 192.168.56.103.

2.3 Clean Installation Script

To resolve package discrepancies and dependency mismatches from previous instances, a hard purge and fresh deployment was executed:

Listing 1: Wazuh Complete Purge and Reinstall

```
# Purge previous installation
sudo apt-get remove --purge wazuh-manager wazuh-indexer wazuh-dashboard wazuh-agent
filebeat -y
sudo rm -rf /var/ossec /etc/wazuh-* /etc/filebeat /var/lib/wazuh-indexer /etc/wazuh-
indexer /var/lib/wazuh-dashboard /etc/wazuh-dashboard

# Pull the single host installation script
curl -s0 https://packages.wazuh.com/4.12/wazuh-install.sh
sudo bash wazuh-install.sh -a -o
```

2.4 Wazuh Indexer Network Configuration Fix

Symptom: OpenSearch (Port 9200) dropped requests from n8n on the Windows host.

Root Cause: The indexer binds strictly to 127.0.0.1 by default.

Resolution Protocol: Edit the config to bind globally:

```
sudo sed -i 's/network.host:./network.host: 0.0.0.0/' /etc/wazuh-indexer/opensearch.yml
sudo systemctl restart wazuh-indexer
```

3 Endpoint Agent & Adversary Simulations

3.1 Deploying Agent on Target Windows Host

Using the Wazuh Manager Web Console (<https://192.168.56.103>), a Windows MSI configuration package was generated with the Manager Host IP 192.168.56.103. The generated PowerShell command was executed as Administrator, followed by:

```
NET START WazuhSvc
```

3.2 Simulation Suite: Tactic-based Alerts Generation

The following payloads are executed inside an administrative PowerShell shell on the target Windows endpoint to trigger specific MITRE ATT&CK rules:

1. Persistence via Hidden Scheduled Tasks (T1053.005)

```
schtasks /create /tn "Windows Security Patch" /tr "powershell.exe -WindowStyle Hidden" /sc onlogon /ru System
schtasks /delete /tn "Windows Security Patch" /f
```

2. Account Deletion Operations (T1531)

```
net user TriageTestUser TemporaryPass123! /add
net user TriageTestUser /delete
```

3. Obfuscated Command Shell Invocation (T1059.001)

```
powershell -EncodedCommand
aQB1AHgAIAAoAE4AZQB3ACoATwBiAGoAZQBjAHQAIABOAGUAdAAuAFcAZQB1AEMAbABpAGUAbgBOACkALgBEAG8AdwBuAGwA
==
```

Note: Decodes to a harmless localhost web client download.

4. Registry Run-Key Persistence (T1547.001)

Severity Level: High (Level 8-9)

Adds a mock persistence registry entry to HKLM\Software\Microsoft\Windows\CurrentVersion\Run. This is monitored closely by Sysmon and Wazuh.

```
# Execution
New-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "TriageTestReg" -Value "calc.exe" -PropertyType String

# Undo / Clean Up
Remove-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "TriageTestReg"
```

5. DNS Query Anomaly (C2 Beaconsing Simulation) (T1071)

Severity Level: Medium-High (Level 7-8)

Simulates Command & Control (C2) communication by generating high-volume DNS queries to a dummy domain (evil.test) from PowerShell.

```
# Execution
1..100 | ForEach-Object {
    Resolve-DnsName -Name "evil.test" -ErrorAction SilentlyContinue
}

# Undo / Clean Up
Clear-DnsClientCache
```

6. Data Staging via File Compression (T1560)

Severity Level: Medium-High (Level 7-8)

Simulates data exfiltration preparation by compressing a system file (hosts) into a ZIP archive. This triggers file creation and access events.

```
# Execution
Compress-Archive -Path "C:\Windows\System32\drivers\etc\hosts" -DestinationPath "C:\Temp\staged_data.zip" -Force

# Undo / Clean Up
Remove-Item "C:\Temp\staged_data.zip" -Force
```

7. Process Injection Simulation (Parent-Child Chain) (T1055)

Severity Level: High (Level 8-9)

Creates a process chain: PowerShell -> CMD -> Notepad. This mimics malware launching child processes to hide its activity.

```
# Execution: Launch CMD from PowerShell, then Notepad from that CMD
Start-Process -FilePath "cmd.exe" -ArgumentList "/c echo InjectionTest" -NoNewWindow -Wait
Start-Process -FilePath "cmd.exe" -ArgumentList "/c start notepad.exe" -NoNewWindow -Wait

# Undo / Clean Up
Get-Process -Name "notepad" | Where-Object { $_.MainWindowHandle -eq 0 } | Stop-Process -Force
```

3.3 Verification Checklist

To ensure the SIEM is properly detecting these simulations, review the Wazuh Dashboard for the following telemetry:

- Check for **Sysmon Event ID 13** (Registry value modifications).
- Check for **Event ID 105** or **1296** (Scheduled Tasks creation).
- Verify **DNS Query Logs** for requests bound to evil.test.
- Confirm **File Creation Events** within the C:\Temp directory.

4 Production n8n Pipeline Implementation

The following specifications perfectly reflect the active production JSON configuration of the n8n orchestration logic.

4.1 Node 1: Schedule Trigger

- **Type:** `n8n-nodes-base.scheduleTrigger`
- **Interval:** Every 50 seconds

4.2 Node 2: Get Wazuh Token

- **Method:** POST
- **URL:** `https://192.168.56.103:55000/security/user/authenticate`
- **Auth Mode:** Basic Access Authentication (`n8nagent`)
- **Options:** Allow Unauthorized Certs: `true`

4.3 Node 3: Get Alerts

- **Method:** GET
- **URL:** `https://192.168.56.103:9200/wazuh-alerts-*/_search?size=50&sort=timestamp:desc`
- **Headers:** Authorization = Bearer `{{ $json.data.token }}`
- **Auth Mode:** Basic Access Authentication (`admin indexer credentials`)

4.4 Node 4: Parse Alerts (JS Code)

This node cleanly extracts OpenSearch telemetry, strips noise, and strictly retains items with a rule level of 3 or higher.

```
// Grab the incoming raw data array cleanly from n8n
const inputItems = $input.all();
if (!inputItems || inputItems.length === 0) return [];

const inputData = inputItems[0].json;
const hits = inputData.hits?.hits || [];

// Map through ALL matching alerts
const alerts = hits.map(hit => {
  const src = hit._source || {};
  return {
    id: hit._id,
    timestamp: src.timestamp,
    agent_name: src.agent?.name || 'unknown',
    agent_ip: src.agent?.ip || 'unknown',
    rule_id: src.rule?.id || 'unknown',
    rule_level: src.rule?.level || 0,
    rule_description: src.rule?.description || 'unknown',
    rule_groups: src.rule?.groups || [],
    mitre_technique: src.rule?.mitre?.technique || [],
    mitre_id: src.rule?.mitre?.id || [],
    mitre_tactic: src.rule?.mitre?.tactic || [],
    full_log: src.full_log || ''
  };
});
```

```

    win_message: src.data?.win?.system?.message || ''
  };
});

// REMOVED .slice(0,1) so all matching alerts are preserved!
// Filters out rule levels below 3 to keep it focused on actionable logs.
const filtered = alerts.filter(a => a.rule_level >= 3);

// Output them as a proper n8n list of distinct items
return filtered.map(alert => ({ json: alert }));

```

4.5 Node 5: Loop Over Items

- **Type:** n8n-nodes-base.splitInBatches
- **Batch Size:** 5 (Splits the processed payload into smaller, manageable chunks for the LLM).

4.6 Node 6: Local AI Triage

- **Method:** POST
- **URL:** http://127.0.0.1:11434/api/generate
- **Headers:** Content-Type = application/json
- **Body Parameters:**
 - **model:** llama3.2
 - **stream:**={{ false }}
 - **prompt:**

```

=You are a SOC L1 analyst. Analyze this Wazuh alert and respond ONLY
in JSON. Agent: {{ $('Loop Over Items').item.json.agent_name }}
IP: {{ $('Loop Over Items').item.json.agent_ip }}
Rule ID: {{ $('Loop Over Items').item.json.rule_id }}
Level: {{ $('Loop Over Items').item.json.rule_level }}/15
Description: {{ $('Loop Over Items').item.json.rule_description }}
MITRE: {{ $('Loop Over Items').item.json.mitre_id ?
$('Loop Over Items').item.json.mitre_id.join(', ') : 'None' }}
Log: {{ $('Loop Over Items').item.json.full_log }}
Respond ONLY in this JSON format: {"verdict": "false_positive" or
"true_positive_auto" or "true_positive_escalate", "confidence": "high"
or "medium" or "low", "reasoning": "2-3 sentences",
"recommended_action": "action"}

```

4.7 Node 7: Parse Verdict (JS Code)

Extracts the Llama 3.2 JSON string and dynamically merges it back with the original Wazuh telemetry using direct back-references.

```

const aiItems = $('Local AI Triage').all();
// Look straight back to the original source node before the loop
const originalAlerts = $('Parse Alerts').all();

return aiItems.map((item, index) => {
  const raw = item.json.response || item.json.message?.content || "";

```

```
// Directly pull the matching original alert data from the root slice
const alertData = originalAlerts[index]?.json || {};

try {
  const jsonMatch = raw.match(/\{[\s\S]*\}/);
  if (!jsonMatch) throw new Error("No JSON found");

  const verdict = JSON.parse(jsonMatch[0]);

  return {
    json: {
      ...verdict,
      ...alertData // Injects missing agent_name, agent_ip, timestamp
    }
  };
} catch(e) {
  return {
    json: {
      verdict: "DEBUG_ERROR",
      confidence: "N/A",
      reasoning: "AI response was blank or malformed.",
      recommended_action: "Check AI Node",
      ...alertData
    }
  };
}
});
```

4.8 Node 8: If (Conditional Escalation Override)

Combinator: OR

- **Condition 1:** `{{ $json.rule_level }} ≥ 5` (Number)
- **Condition 2:** `{{ $json.verdict }} == true_positive_escalate` (String)

4.9 Node 9: HTTP Request (Discord Webhook)

Sends a strictly formatted payload to Discord if the IF Node branch evaluates to True.

- **Method:** POST
- **Payload Body:**

```
==SOC ALERT - {{ $json.verdict }}==
**Confidence:** {{ $json.confidence }}
**Reasoning:** {{ $json.reasoning }}
**Action:** {{ $json.recommended_action }}
**Device Name:** {{ $json.agent_name || 'N/A' }}
**Device IP:** {{ $json.agent_ip || 'N/A' }}
**Source IP:** {{ $json.source_ip || 'N/A' }}
**Triggered Time:** {{ $json.timestamp || 'N/A' }}
```


5 Troubleshooting Registry & Stabilizations

5.1 The Metadata Field Depletion (Empty Fields / N/A Bug)

Problem: Alerts arriving in Discord had empty telemetry fields (Device Name: N/A, Device IP: N/A) even though the AI verdict and reasoning were successfully generated.

Root Cause Analysis: The Parse Verdict node was returning only the parsed JSON response from the LLM, dropping the original alert metadata fields from the n8n data stream.

Remediation: The code inside the Parse Verdict node was entirely re-written to capture `$( 'Parse Alerts' ).all()` and map the indices. The spread operator (`...alertData`) merges original telemetry with the AI verdict, solving the context loss.

5.2 The 1-Item Batch Strangle

Problem: When running an exploit chain simulating dozens of alerts, only one alert made it to Discord.

Root Cause Analysis: The core ingestion node was returning the entire batch wrapped inside a top-level array. Additionally, early pipeline designs included a `.slice(0, 1)` limitation.

Remediation: Removed `.slice(0,1)` from the Parse Alerts node. Introduced a `splitInBatches` ("Loop Over Items") node with a `batchSize` of 5. This iterates properly over all identified events while managing LLM processing load.

5.3 Ollama Streaming Crash (Go Unmarshal String Errors)

Problem: The Parse Verdict node threw parser failures logging "AI response was blank or malformed." Concurrently, Ollama logged 400 "cannot unmarshal string into Go struct field stream of type bool".

Root Cause Analysis: Sending "stream": "false" as a static text string caused the Go-based backend to reject the request payload.

Remediation: Changed the stream parameter inside the n8n HTTP request body to an expression evaluated as `={{ false }}`. This strictly casts the type to a boolean, properly disabling token-streaming and returning a single JSON block.

5.4 Llama 3.2 AD Account Deletion Hallucination

Problem: During a simulated account deletion event (MITRE T1531 / Level 8), the local LLM returned a false-positive verdict: "This is a legitimate action taken by the system administrator...". Under the old logic, this critical event was dropped.

Root Cause Analysis: The workflow relied entirely on the LLM's classification to route alerts, silencing valid alerts on AI hallucination.

Remediation: Reconfigured the IF Node using an **OR** logical routing operator. High-severity alerts (`rule_level ≥ 5`) now trigger a hard override bypass, escalating directly to Discord regardless of AI tag.

5.5 Google Gemini API Constraints

Problem: Initial drafts intended to use the Google Gemini API (free tier) for the triage logic. This encountered continuous Rate limit errors (429) and Server overload errors (503).

Remediation: The architecture was hard-shifted to a strictly localized inference engine using Ollama running llama3.2. This entirely eliminated API rate limiting, external service dependencies, and data-residency compliance risks.